CareerNet — Sprint 2 Contribution Log

Student: Krish Patel
Course: SOEN 341 — Software Process
Project: CareerNet
Sprint: Sprint 2 (October 8–26, 2026)

Work Session 1 — October 9, 2026

Task: KR-1 — Profile Page (Mock Data First)

Time Spent: 3 Hours

Work Completed:

- Updated my local Git repository and created the feature/KR-1-profile-page branch.
- Created src/types/profile.ts to define the profile data fields.
- Created src/api/profile.ts with temporary mock API functions.
- Implemented ProfilePage.tsx for creating, viewing, saving, and editing a user profile.
- Added input validation for required full name, a maximum of 2,000 summary characters, and up to 30 skills.
- Added the protected /profile route in App.tsx.
- Added a **My Profile** link to the navigation bar.
- Manually tested the profile page using mock login and mock profile data.
- Committed and pushed the changes to my feature branch.
- Created Pull Request #67 and requested Sayf's review.

Testing and Validation:

Tested profile creation, editing, form validation, saving, and navigation between pages.
Confirmed that the mock profile remains available while navigating in the running application.

Challenges and Solutions:

Encountered unresolved Git index conflicts involving Sprint 1 documentation before creating the feature branch. Resolved the local repository state before starting KR-1.

The shared database login was not available during testing, so I used CareerNet's existing mock authentication to test the frontend without depending on the unfinished backend integration.

GitHub References:

- Task Issue: https://github.com/Sayf-01/SOEN341-2026/issues/40
- Pull Request: https://github.com/Sayf-01/SOEN341-2026/pull/67
- Branch: feature/KR-1-profile-page

Current Status: Implementation and manual testing completed; awaiting Pull Request review and merge.

Work Session 2 — October 11, 2026

Task: KR-2 — Resume Section (Mock Data First)

Time Spent: 2 Hours

Work Completed:

- Created the feature/KR-2-resume-section branch based on my existing KR-1 Profile Page branch.
- Created src/types/resume.ts to define the resume data fields, including ID, filename, content type, file size, and upload date.
- Created src/constants/resumeRules.ts to define the allowed resume formats (PDF, DOC, and DOCX) and the maximum file size of 5 MB.
- Created src/api/resumes.ts with mock API functions for uploading, listing, replacing, downloading, and deleting resumes.
- Implemented ResumeSection.tsx to display uploaded resumes with their filenames, file sizes, and upload dates.
- Added Upload, Replace, Download, and Delete buttons to manage resume files.
- Added confirmation before deleting a resume.
- Added loading, success, error, and empty-state messages.
- Integrated the Resume Section into the existing Profile Page.
- Improved the Delete button's readability by changing its text color to white.
- Manually tested the Resume Section using mock data.
- Committed and pushed the changes to my KR-2 feature branch.

Testing and Validation:

Manually tested uploading valid PDF and Word documents, displaying uploaded resumes, downloading files, replacing existing resumes, and deleting resumes with confirmation.

Also tested invalid file types, files exceeding the maximum size limit, and the empty-state message after deleting resumes. Confirmed that the Resume Section appears correctly on the Profile Page and that the main functionality works as expected.

Challenges and Solutions:

The resume upload and management endpoints were not yet implemented in the backend. To avoid delaying frontend development, I used mock API functions with temporary in-memory storage.

I also noticed that the Delete button's original text color was difficult to read against the red background, so I changed the text color to white for better visibility.

The team also introduced a new branching workflow in which task branches are merged into user-story branches instead of directly into main. I kept KR-2 on its own feature branch to follow this workflow.

GitHub References:

- Task Issue: https://github.com/Sayf-01/SOEN341-2026/issues/41
- Related User Story: https://github.com/Sayf-01/SOEN341-2026/issues/16
- Feature Branch: https://github.com/Sayf-01/SOEN341-2026/tree/feature/KR-2-resume-section
- Pull Request: Not created yet; awaiting confirmation of the user-story branch.

Current Status: Implementation and manual testing completed. Changes committed and pushed to the feature branch. Awaiting Pull Request creation and review under the new user-story branching workflow.

Work Session 3 — October 11, 2026

Task: KR-3 — Apply Dialog with Confirmation (Mock Data First)

Time Spent: 2 Hours

Work Completed:

- Created the feature/KR-3-apply-dialog branch based on my KR-2 branch.
- Created src/api/applications.ts with a mock applyToJob() function for submitting job applications.
- Added validation for selected resumes and cover notes.
- Implemented duplicate application detection with a 409-style error.
- Created src/components/ApplyDialog.tsx as a reusable job application component.

- Added a resume dropdown connected to the existing getResumes() function from KR-2.
- Added an optional cover note field with a maximum length of 1,000 characters.
- Added Submit Application and Cancel buttons.
- Implemented a confirmation message after successful submission, including a link to My Applications.
- Added a message directing users to upload a resume if they do not have one.
- Implemented Escape-key support and automatic focus for the resume dropdown.
- Temporarily integrated the Apply Dialog into the Profile Page for testing.
- Removed the temporary testing code after validation.
- Committed and pushed the KR-3 files to GitHub.

Testing and Validation:

Ran npm run build successfully.

Manually tested opening and closing the dialog, selecting a resume, submitting an application, and displaying the confirmation message.

Also tested submitting a duplicate application, attempting to apply without an uploaded resume, the cover note character limit, the Cancel button, the Escape key, and displaying multiple resumes in the dropdown.

Confirmed that the expected functionality worked before removing the temporary testing component.

Challenges and Solutions:

Encountered a TypeScript error involving the public status: number constructor parameter in the custom ApplicationError class. Resolved the issue by declaring status as a separate class property and assigning it inside the constructor. The project built successfully afterward.

The Job Detail Page was not yet available from Dany's work, so I temporarily displayed the Apply Dialog on the Profile Page to perform manual testing without depending on another feature.

GitHub References:

- Task Issue: https://github.com/Sayf-01/SOEN341-2026/issues/49
- Feature Branch: https://github.com/Sayf-01/SOEN341-2026/tree/feature/KR-3-apply-dialog

- Commit: 4a1090a
- Pull Request: Not created yet; awaiting confirmation of the team's user-story branching workflow.

Current Status: Implementation, build verification, and manual testing completed. Changes committed and pushed. Awaiting Pull Request review and integration with Dany's Job Detail Page.

Work Session 4 — October 11, 2026

Task: KR-4 — Recruiter Dashboard (Mock Data First)

Time Spent: 3 Hours

Work Completed:

- Updated my local main branch with the latest GitHub changes.
- Created the feature/KR-4-recruiter-dashboard branch from main to keep this feature independent of KR-1, KR-2, and KR-3.
- Created src/api/recruiter.ts with mock API functions for creating job postings and retrieving applications for a specific job.
- Implemented RecruiterDashboardPage.tsx with a job posting creation form and an applications section.
- Added five required fields: job title, company, location, category, and description.
- Added validation messages for missing or invalid form information.
- Added the six agreed job categories.
- Implemented a success message and job details display after creating a posting.
- Added a View Applications button, loading state, and empty-state message.
- Added the protected /recruiter route in App.tsx.
- Updated ProtectedRoute.tsx to support role-based access.
- Added a Recruiter Dashboard navigation link visible to recruiter accounts.
- Temporarily modified mock authentication to test the dashboard using a recruiter account.
- Removed the temporary testing modification before committing.
- Committed and pushed the changes to GitHub.

Testing and Validation:

Ran npm run build successfully.

Manually tested recruiter login, access to the Recruiter Dashboard, required-field validation, successful job posting, and display of the created job's information.

Also tested the View Applications button and confirmed that the message "No applications yet" appears when a posting has no applicants.

Verified that the Recruiter Dashboard link is hidden for job seekers and that a job seeker attempting to access /recruiter is redirected to the normal dashboard.

Confirmed that the temporary mock authentication changes were removed before committing.

Challenges and Solutions:

The recruiter backend endpoints were not yet available, so I implemented mock API functions to develop and test the interface independently.

The existing mock login assigned all users the Job Seeker role. I temporarily added a recruiter test account to validate role-based navigation and access, then restored the original authentication behavior.

The shared job categories and status badge components from Dany were not yet integrated. I used the agreed categories and a simple status display temporarily.

GitHub References:

- Task Issue: https://github.com/Sayf-01/SOEN341-2026/issues/53
- Feature Branch: https://github.com/Sayf-01/SOEN341-2026/tree/feature/KR-4-recruiter-dashboard
- Commit: https://github.com/Sayf-01/SOEN341-2026/commit/f4c9223
- Pull Request: Not created yet; awaiting confirmation of the user-story branching workflow.

Current Status: Implementation, manual testing, and production build verification completed. Changes committed and pushed to GitHub. Awaiting Pull Request creation, review, and eventual backend integration.